CUDA Ray Tracing

Implementation and Performance Analysis

Adithya Kaushik
CS23B002

Contents

1 Implementation **3**

1.1 CPU Reference Implementation 3

1.2 CUDA Port and Mapping Strategy 3

1.2.1 Pixel-to-Thread Mapping 3

1.2.2 Grid and Block Configuration 3

1.3 Lighting and Shadows 4

1.4 Multi-Bounce Reflections 4

1.4.1 Iterative Reflection Computation 4

1.4.2 Termination Conditions 4

1.4.3 Other changes 5

2 Experimental Setup **5**

2.1 GPU Hardware and CUDA Environment 5

2.1.1 Scene Configurations 6

3 Performance Analysis **6**

3.1 Scaling with Image Resolution 6

3.2 Scaling with Reflection Depth 6

3.3 Effect of CUDA Block Size 8

4 Bounding Volume Hierarchy (BVH) - Algorithmic reduction of intersection tests **9**

4.0.1 Construction 9

4.0.2 Traversal and Intersection Testing 10

4.0.3 Difficulties while coding this up 10

5 GPU Profiling and Bottleneck Analysis **11**

5.1 Nsight Systems Analysis 11

5.2 Nsight Compute Analysis 11

5.3 Different Scenes 11

5.4 Warp Instructions and Divergence 12

5.5 Occupancy 14

5.6 Register Pressure 14
5.7 Memory Behaviour 15
5.8 Identified Bottlenecks 16

6 Optimizations **17**
6.0.1 Performance Comparison 17
6.0.2 BVH Optimizations 18
6.1 Other Optimization Attempts 19
6.2 Unsuccessful Optimizations and Analysis 21

7 Rendered Outputs **23**
7.1 Baseline CUDA Rendering 23
7.2 Multi-Bounce Reflections 23

8 Conclusion **24**

1 Implementation

1.1 CPU Reference Implementation

The CPU reference implementation was single threaded and did not implement reflections or reflective objects. The assumptions below that were made for the starter code are also persisted throughout the GPU code:

- Single Light Source.
- Camera position as defined in the starter code is fixed even in the GPU implementation.
- Floor is still considered as a checkerboard sphere.

An important point to note is that the starter implementation only considered the nearest intersection point for a ray of light. In my GPU implementation, I have considered both intersection points (and their relevant distance parameter t). If the camera is ever changed to be inside an object, then the farther intersection point may become relevant.

To prevent self intersection in finding shadows or in reflected rays, the generated rays are shifted along the normal of the relevant intersected object by some small amount ϵ.

1.2 CUDA Port and Mapping Strategy

The predominant difference between CPU and GPU implementation for the ray tracer is that the CPU code opts to loop over every single pixel sequentially, whereas the GPU implementation can parallelize that computation.

1.2.1 Pixel-to-Thread Mapping

In my code, I assign one thread to one pixel. That thread generates one ray and follows all the relevant computations for that ray. Finally, the colour of that pixel is the result of all accumulated colours and shadows.

Consider an image of resolution 800×600, the CPU code does 480000 iterations before writing the image, the GPU code parallelizes this.

1.2.2 Grid and Block Configuration

Before any optimizations, the grid and block configurations are as follows:

```
#define threads_per_block 16

int blocks_x = (W + threads_per_block - 1) / threads_per_block;
int blocks_y = (H + threads_per_block - 1) / threads_per_block;

dim3 blocks(blocks_x, blocks_y, 1);
dim3 threads(threads_per_block, threads_per_block);
```

Notice that the x-dimension gets the width and the y-dimension gets the height, consistent with row major ordering of threads on a GPU thus improving memory coalescing. Consequently, using Horner's rule we arrive at the flat thread index:

```
int py = blockIdx.y * blockDim.y + threadIdx.y;
int px = blockIdx.x * blockDim.x + threadIdx.x;

int rgb_idx = (py * W + px) * 3;
```

1.3 Lighting and Shadows

Lighting is split into two parts for better realism:

- Ambient Lighting: Set to 0.15, no pixel is ever totally dark.
- Diffuse Lighting: Depends on the dot product between the vector pointing from the point of intersection of the ray to the light, and the normal at the point of intersection. It is clipped at 0. If the normal and light vector point in opposite directions, that point is completely in shadow in the absence of ambient light.

A small but important optimization was made here, I created a new function to specifically find if the a ray generated from a point, directed towards the light source intersected with an object before reaching the light source. In the starter code, full ray collision logic was employed, but this does more work than necessary and loops through every single object in the scene even if we have a relevant shadow collision. In my function, if such a collision is found, we short-circuit and return "true" immediately.

1.4 Multi-Bounce Reflections

Added an attribute "reflectivity" here to denote how reflective an object is.

1.4.1 Iterative Reflection Computation

Here I employed two tactics, the first was inefficient compared to the second.

Initially, in DFS fashion, I thought to maintain a stack to keep track of all the objects that a ray collided with on its path upto maximum bounce depth and then compute the colour accumulation. It was also difficult to handle the case when the ray bounced towards the sky before bounce depth was reached.

Then I switched to using an accumulator that considers a running reflectivity to compute the intensity of reflected colour as the ray bounces more:

```
ColourSoFar += obj.colour * brightness_of_object * (1 - obj.reflectivity) * residue
residue *= obj.reflectivity
```

Let r be the reflectivity of an object. When a ray of light hits the object, only a $1 - r$ fraction of the object's base colour is persisted in the pixel, the rest comes from the colours of the ray's subsequent bounces.

1.4.2 Termination Conditions

Two termination conditions were used, as advised in the problem statement, and one extra which is necessary:

- **Maximum bounce depth**: This particularly takes care of infinite bounces in the case of two perfectly reflective mirrors facing each other. It also allows compute to be conserved.

- **Energy threshold**: If the residual colour fraction coming from remaining bounces of a ray falls below a certain threshold, we do not need to compute it, since its effects would be inconsequential.
- **Ray misses scene**: If, before the bounce depth or energy threshold is reached, a ray misses all objects in the scene and bounces towards the sky, we can terminate computation.

1.4.3 Other changes

A few more changes were made in the form of how input is taken and cudaEvent timers:

- **Taking input**: I modified the code to allow input to be taken from a file so that the main logic remains easy to read.
- **CUDA Events**: Created a timer function that takes in the ray trace kernel as a callable function and finds the average runtime of the kernel after any required number of rounds. In addition, five warm-up iterations are executed before timing to reduce cold-start effects and obtain more representative measurements of repeated kernel execution.

2 Experimental Setup

2.1 GPU Hardware and CUDA Environment

All code was profiled on Google Colab NVIDIA T4 GPUs.

Below are the specifications: `nvidia-smi -q`

- Compute Capability: 7.5
- Streaming Multiprocessors (SMs): 40
- CUDA Cores: 2560
- Global Memory: 16 GB GDDR6

The CUDA source code was compiled using the following flags:

```
flags = ['-std=c++17', '-O3', '-lineinfo', '-Xptxas=-v', f'-arch=sm_{arch}']
```

What each flag does:

- "-O3": enables aggressive host-code optimizations such as inlining, loop unrolling and vectorization. NVCC already uses high-level optimizations for GPU code by default.
- "-lineinfo": adds source-line information for profiling without compiling in device debug mode.
- "-Xptxas=-v": Makes the ptx assembler print important code generation statistics, such as register usage, shared memory usage and constant memory usage.
- "-arch=sm_arch": Tells NVCC which GPU architecture to compile for; it does not itself detect the GPU. The Python variable `arch` is set to 75 for the Tesla T4, corresponding to compute capability 7.5.

Kernel execution time was measured using CUDA events. Each benchmark consisted of 5 warm-up launches followed by 20 measured launches, with the average execution time reported in milliseconds.

2.1.1 Scene Configurations

All principal scene-complexity experiments used an image resolution of 800×600 pixels, with scenes containing 10, 100, 500, 1000, and 5000 spheres.

These timings measure GPU kernel execution and exclude host-side scene construction, BVH construction, memory allocation, host-device transfers, and image file output.

3 Performance Analysis

NVIDIA Nsight Systems was used to inspect kernel execution and CUDA API activity, while NVIDIA Nsight Compute was used to collect hardware performance metrics, including instruction counts, occupancy, warp utilization, register usage, and memory-system behaviour.

3.1 Scaling with Image Resolution

This should look roughly linear, with a fixed aspect ratio of 4:3. I checked two different scene complexities, namely 100 spheres and 5000 spheres. The block dimensions were 16×16.

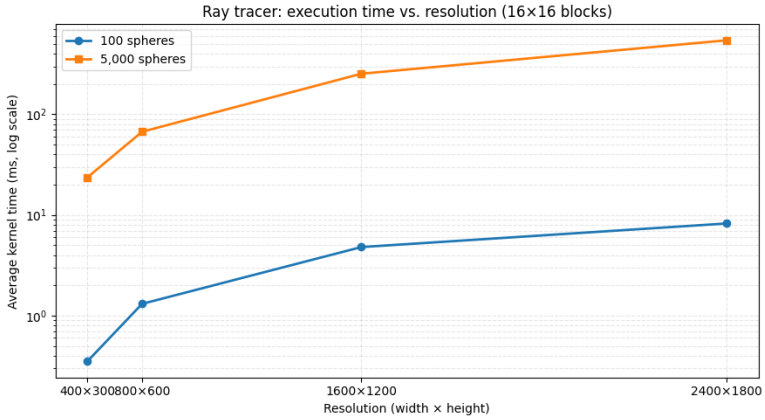


Figure 1: Variation in kernel execution time with image resolution for scenes containing 100 and 5000 spheres, with a fixed aspect ratio of 4:3.

For brute-force tracing, if the maximum reflection depth is fixed, the total work is $O(NWH)$, where N is the number of spheres/objects, W is the image width, and H is its height. More generally, with a maximum of D reflections, the worst-case work is $O(WHN(D+1))$, since each additional ray can require another scan through the scene (including shadow intersection checks). This is a bound on total work, not necessarily on GPU execution time, since many pixels are processed in parallel. With a BVH, intersection queries can be much cheaper in favourable cases, although the worst-case cost per query is still $O(N)$. I have used logarithmic scale to fit things properly on the screen.

3.2 Scaling with Reflection Depth

To study the effect of reflection depth on kernel execution time, the maximum number of permitted reflections was varied while keeping the scene complexity fixed at 1000 spheres, the image resolution at 800×600 and the block dimensions at 16×16.

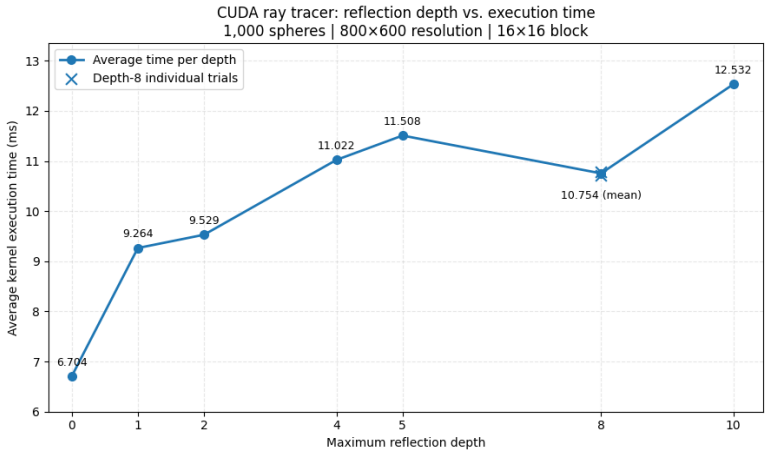


Figure 2: Variation in kernel execution time with maximum reflection depth for a scene containing 1000 spheres. (Highly unoptimized BVH)

As shown in Figure 2, increasing the maximum reflection depth generally increases kernel execution time. At a depth of 0, where no reflected rays are traced, the average execution time was 6.704 ms. This increased to 9.264 ms at depth 1 and 11.508 ms at depth 5. At a maximum depth of 10, the execution time reached 12.532 ms, representing an approximately 87% increase over the depth-0 implementation.

However, the increase in execution time was not strictly monotonic. In particular, the two measurements at depth 8 were 10.718 ms and 10.790 ms, both lower than the measured time at depth 5.

An important distinction is that the maximum reflection depth only establishes an upper bound on the number of reflections traced per pixel. A reflected ray may terminate earlier if it does not intersect another object, encounters a non-reflective surface, or its remaining contribution falls below the termination threshold. Consequently, increasing the maximum depth does not necessarily increase the amount of computation proportionally.

The observed decrease at depth 8 cannot be explained conclusively from these timings alone and may also reflect measurement variability. Still the overall results demonstrate that supporting additional reflections introduces computational overhead, while early ray termination prevents execution time from scaling directly with the maximum permitted depth.

Note that an energy threshold was instated, meaning that some threads could exit early if they didn't meet the threshold.

3.3 Effect of CUDA Block Size

Varying CUDA block dimensions between 64 and 512 threads per block produced relatively small changes in execution time compared with the effects of scene complexity and BVH acceleration. For the 5,000-sphere scene, measured runtimes ranged from approximately 65.9 ms to 72.7 ms. Smaller blocks performed somewhat better for this scene, although additional measurements would be required to establish what is an optimal configuration. Overall, the results suggest that reducing ray-object intersection work through algorithmic acceleration has a substantially greater impact than tuning the block configuration.

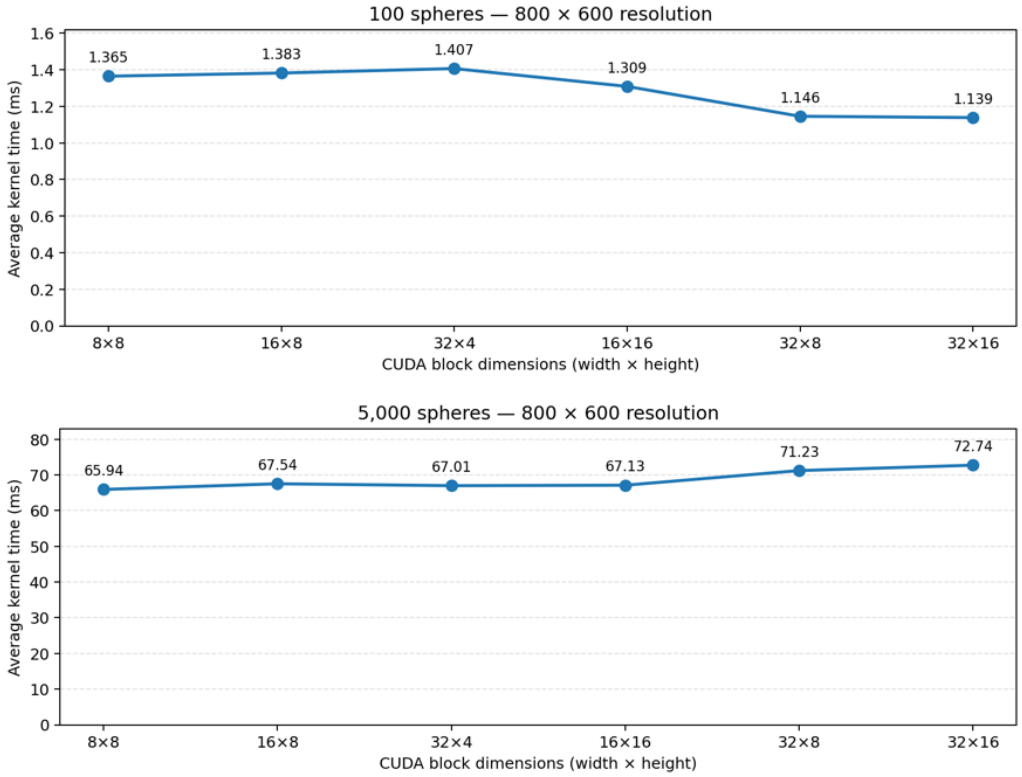


Figure 3: Effect of CUDA thread block dimensions on kernel execution time for 100 and 5000 spheres at 800×600 resolution. Both plots use linear scales, and repeated measurements for the 32×8 configuration are averaged.

4 Bounding Volume Hierarchy (BVH) - Algorithmic reduction of intersection tests

Below is a rough image of what a BVH Tree looks like:

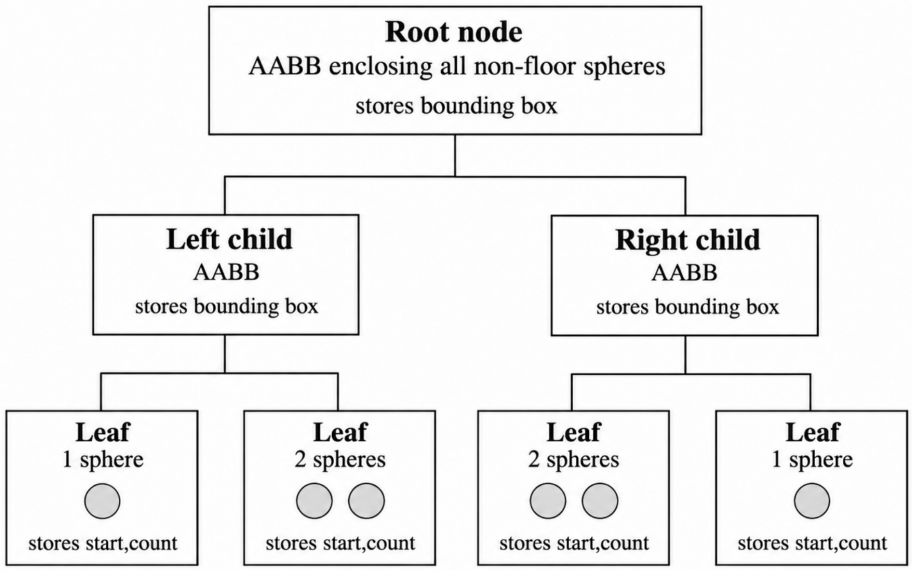


Figure 4: BVH Tree Example Graphic

4.0.1 Construction

The BVH is constructed recursively on the CPU. A Bounding Volume Hierarchy (BVH) is a binary tree in which every node stores an Axis-Aligned Bounding Box (AABB) enclosing all objects belonging to its subtree. The objective is to organize the objects spatially such that large groups of objects can be rejected during ray intersection testing without individually testing each object.

Construction begins by computing an AABB enclosing all objects in the scene. At each internal node, the longest axis of the enclosing AABB is selected as the splitting axis.

The objects are then partitioned into two approximately equal groups according to the median of their centre coordinates along this axis. This is implemented using `std::nth_element`, which rearranges the object indices without requiring a complete sort.

The same procedure is applied recursively to both partitions until the number of objects in a node is at most the chosen leaf size.

In the final optimized implementation, the leaf size is set to one, meaning that each leaf contains exactly one sphere. Consequently, for n non-floor spheres, the resulting full binary tree contains n leaves and $n - 1$ internal nodes, giving a total of $2n - 1$ nodes.

Once construction is complete, the sphere data is reordered according to the ordering produced by the BVH partitioning procedure. This ensures that objects belonging to a leaf occupy contiguous positions in memory.

Furthermore, a separate `float4` array stores the centre coordinates and squared radius of each sphere, reducing the amount of object data required during intersection testing. The constructed BVH and reordered geometry are subsequently transferred to GPU memory.

4.0.2 Traversal and Intersection Testing

The recursive spatial partitioning offered by the BVH substantially reduces the number of primitive intersection tests required for ray tracing. In the brute-force implementation, every ray must test all n spheres to determine its nearest intersection. With a BVH, entire subtrees can be eliminated whenever the ray does not intersect their bounding boxes.

Traversal is performed iteratively on the GPU using an explicit stack maintained independently by each CUDA thread. Initially, the root node is pushed onto the stack. At each iteration, a node is popped and its bounding box is considered for intersection. For internal nodes, the ray is tested against the bounding boxes of both children. Only intersected children are added to the stack.

When both children are intersected, the farther child is pushed first and the nearer child second. Since the stack follows a Last-In-First-Out (LIFO) ordering, the nearer child is processed first. This ordering increases the possibility of discovering a close intersection early, allowing bounding boxes lying beyond the current closest intersection to be rejected.

Ray-AABB intersection is implemented using the slab method. For each coordinate axis, the interval of ray parameters corresponding to the bounding box is computed. The intersection of these three intervals determines whether the ray intersects the box. To avoid repeated floating-point divisions during traversal, the reciprocal of each ray direction component is precomputed once per ray. Subsequent bounding-box tests therefore use multiplication instead of division.

When a leaf node is reached, the ray is tested against its associated sphere using the quadratic ray-sphere intersection equation. The closest positive intersection is maintained throughout traversal. For shadow rays, traversal can instead terminate immediately upon finding an occluding object between the surface point and the light source. The same BVH traversal procedure is reused for primary rays, reflected rays, and shadow rays, with each ray maintaining its own traversal state.

In favourable cases, a well-constructed BVH allows nearest-hit queries to examine only a small fraction of the scene, often exhibiting traversal behaviour close to $O(\log n)$.

However, this is not a strict worst-case guarantee: overlapping bounding boxes may require traversal of multiple subtrees, and the worst-case complexity remains $O(n)$.

Practical performance depends strongly on the quality of spatial partitioning, object distribution, and ray coherence.

Unlike brute-force intersection testing, BVH traversal introduces irregular memory accesses and warp divergence because different rays may follow different paths through the tree.

Despite these costs, the reduction in intersection tests provides substantial performance improvements for big enough scenes.

4.0.3 Difficulties while coding this up

- Initially, `intersectAABB()` received `tClosest, tFarthest` by value. The computed entry distances weren't propagated to the caller, and my near-first child ordering did not function as intended.
- Excluding the floor required special handling. Removing the large floor sphere from the BVH improved the hierarchy's structure, but we had to preserve its intersection tests for primary, reflected, and shadow rays.
- Incorrect quadratic formula change. While trying to reduce arithmetic with the half-b ray sphere intersection formula, I initially mixed coefficients from the standard and simplified forms, leading to incorrect intersection calculations.

5 GPU Profiling and Bottleneck Analysis

5.1 Nsight Systems Analysis

This analysis was used minimally in lieu of consistent albeit slightly slower CUDA event timers.

Another reason for avoiding Nsight Systems was the large number of files produced per run for a single scene. Importantly, since CUDA event timers were employed, Nsight Compute Analysis became more important.

In any case, below is the syntax used:

```
!nsys profile --trace=cuda --sample=none -o nsys_{n} ./raytracer < raytracer_scene.txt

!nsys stats \
    --report cuda_gpu_kern_sum \
    --report cuda_api_sum \
    --report cuda_gpu_mem_time_sum \
    nsys_{n}.nsys-rep
```

5.2 Nsight Compute Analysis

This offers a huge number of valuable statistics that are discussed below. The following is the command used to execute ncu:

```
!ncu --set full \
    --kernel-name regex:ray_trace \
    --launch-skip 5 \
    --launch-count 1 \
    -o ncu_{n}\
    ./raytracer < input
```

We profile only one kernel at a time, in particular there is only one kernel in the ray-tracer. We skip the first five launches in our warmup. "–set full" prints all statistics.

A few runs reported an L2 cache hit rate above 100%. This is not a physically meaningful hit rate. NVIDIA documents that out-of-range ratios can occur when Nsight Compute collects the underlying counters across different kernel replay passes, particularly for short or variable workloads. I therefore did not use these values to draw conclusions about cache behaviour.

The following analysis uses Nsight Compute profiles of the 5000-sphere scene on an NVIDIA T4, without fast math. Each implementation was profiled once; execution times are averages from separate, unprofiled benchmark runs. Some measurements in the profiling discussion came from an earlier benchmark session, while the main optimization comparisons use a later session. This explains small differences in absolute timings; speedups are calculated from measurements within the same session.

5.3 Different Scenes

In my scene-scaling experiments, smaller scenes were prefixes of larger ones, so that experiments were largely consistent. For the later large-scene comparison, however, I used two different spatial arrangements, not the same scene with one extra sphere. This matters much more for BVH because the traversal paths depend on the distribution of objects.

Scene 5000: 5000 spheres with randomly placed, sometimes overlapping objects. Their bounding boxes can overlap, causing more BVH traversal work.

Scene 5001: 5001 spheres arranged roughly along the floor, without overlapping each other. Their bounding boxes are generally better separated, allowing BVH traversal to reject larger groups of objects sooner.

Thus, any timing difference between these two scenes reflects a change in spatial arrangement and not the effect of adding just one sphere. In brute force, every nearest-hit ray still scans the scene, so the arrangement generally has a smaller effect.

Note that camera and light positions are also crucial for render times as they determine which objects are visible and in what amount.

5.4 Warp Instructions and Divergence

Warp divergence occurs when threads within the same warp take different control-flow paths. The paths are executed with the threads not following a particular path masked off, so some warp instructions run with fewer than 32 active lanes. This wastes part of the available execution capacity and can increase the total number of instructions needed to complete a warp's work. In this ray tracer, it is especially relevant because different rays can visit different parts of the BVH or terminate at different times.

Warp divergence is influenced by:

- Ray Coherence: We want coherent rays, or those rays with similar origins and directions to be part of the same warp. If they are, they tend to execute the same instructions for longer and exit around the same time.
- BVH Traversal Paths: Some rays may visit the left subtree and some the right at a given node, depending on their intersection with the children's bounding boxes.
- Shadow Ray Termination: This is a similar problem, since the same object intersection logic is applied here. Shadow rays may terminate faster if they hit an obstacle before the light source, causing warp divergence.
- Reflection Depth: The larger the maximum reflection depth, the longer some rays in a warp may persist compared to their terminated counterparts.

For the 5000-sphere scene, brute force averaged 21.48 active threads per warp instruction, compared with 11.06 for basic BVH and 9.22 for optimized BVH, out of 32 possible threads. The lower BVH lane participation is consistent with neighboring rays following different traversal paths and completing different amounts of work.

Nevertheless, optimized BVH reduced executed warp instructions from 969 million to 372 million, a 61.6% reduction. In the latest benchmark session, runtime decreased from 8.315 ms to 3.748 ms, a 54.9% reduction. Thus, eliminating traversal work outweighed the poorer lane participation.

The different 5001-sphere scene showed greater BVH lane participation: brute force averaged 21.65 active threads, basic BVH 14.13, and optimized BVH 12.85. Compared with the 5000 sphere scene, optimized BVH executed 41.1% fewer warp instructions, decreasing from 372 million to 219 million, while runtime fell from 3.748 ms to 1.887 ms. Basic BVH executed 669 million instructions on this scene, so optimized BVH reduced its instruction count by approximately 67.2%.

Although the sphere counts were almost identical, the different scene substantially changed BVH work and lane participation. Brute-force lane participation remained nearly unchanged, and its runtime decreased by only 6.9%, compared with 49.7% for optimized BVH. These results demonstrate greater scene sensitivity for BVH, rather than complete scene independence for brute force. The traversal and axis-split experiments targeted work reduction.

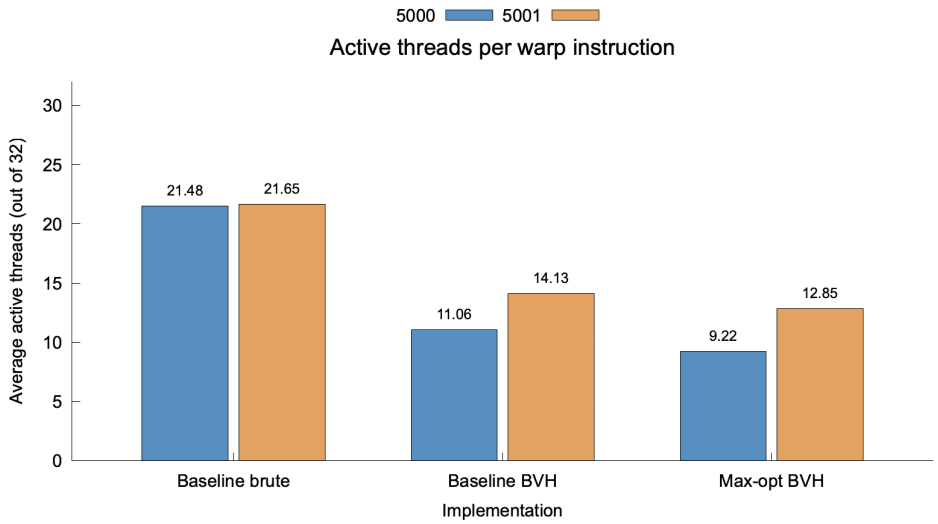


Figure 5: Number of active threads in a warp

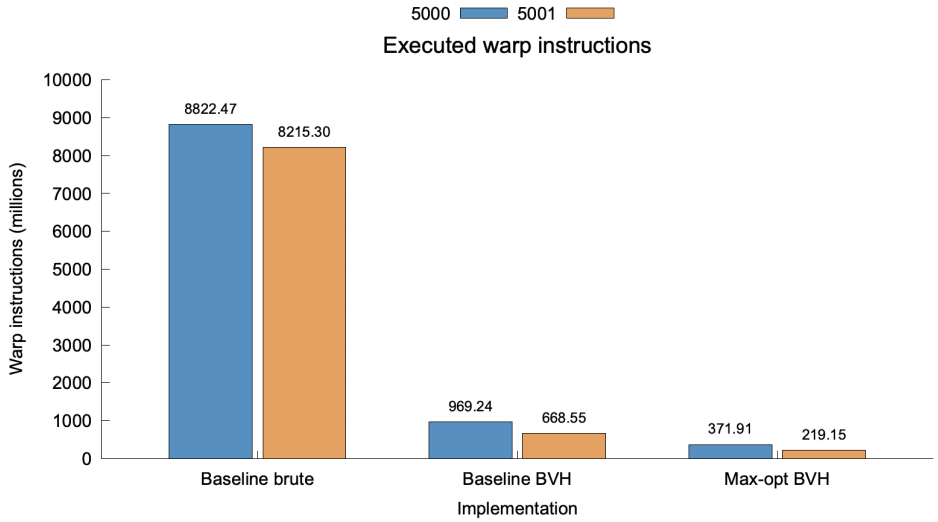


Figure 6: Number of instructions executed

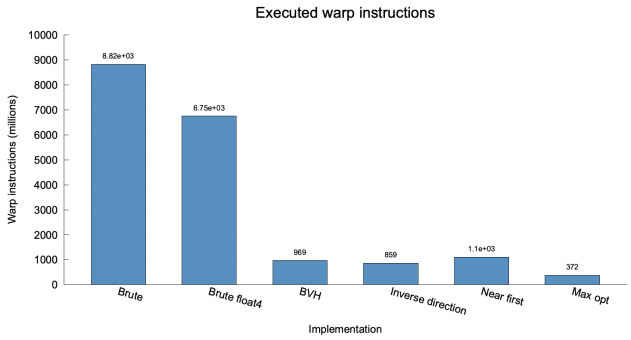


Figure 7: Executed instructions for the additional optimization variants

5.5 Occupancy

An SM keeps many warps resident so that it can execute another warp while one waits for data or an instruction dependency. Occupancy measures how many of the available resident warp slots are occupied. It does not measure how many threads within each warp are participating, or how many warps are ready to execute. A resident warp still contributes to occupancy while stalled.

Occupancy depends on register allocation, block size, shared-memory usage, and workload balance. For scene 5000, basic BVH achieved 61.75% occupancy against a 75% theoretical ceiling, while optimized BVH achieved 74.23% against a 100% ceiling. For scene 5001, achieved occupancy was 60.91% and 74.89%, respectively, with unchanged theoretical ceilings. Thus, optimized BVH maintained higher occupancy on both scenes, but changing the scene had little effect on occupancy.

Higher occupancy provides more opportunities to hide latency, but does not guarantee that resident warps are ready to execute. For scene 5000, eligible warps per scheduler per cycle decreased from 1.098 for basic BVH to 0.872 for optimized BVH, despite the occupancy increase. scene 5001 showed the same pattern: 1.147 for basic BVH versus 0.967 for optimized BVH.

Optimized BVH's runtime decreased by 49.7% between scenes 5000 and 5001, while its occupancy increased by only 0.66 percentage points. The substantial performance improvement is therefore better explained by reduced instruction count and improved lane participation and memory locality than by occupancy alone. These results motivate evaluating block size and register usage alongside instruction dependencies and execution time.

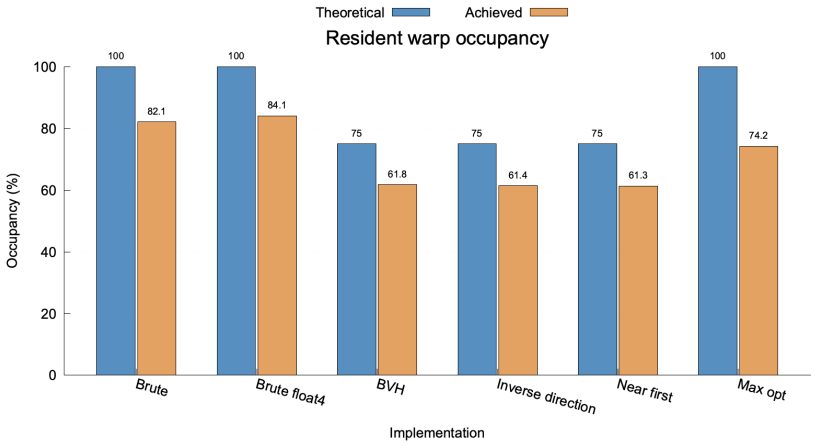


Figure 8: Warp Occupancy

5.6 Register Pressure

Register pressure depends on live values, intermediate calculations, compiler scheduling and traversal state in BVH simultaneously. Basic BVH used 70 registers per thread, rounded to 72 allocated registers; optimized BVH used and allocated 56.

With 256 threads per block and 65,536 registers per SM, basic BVH requires 18,432 registers per block, permitting three resident blocks. Optimized BVH requires 14,336, permitting four. This explains the increase in theoretical occupancy from 75% to 100%.

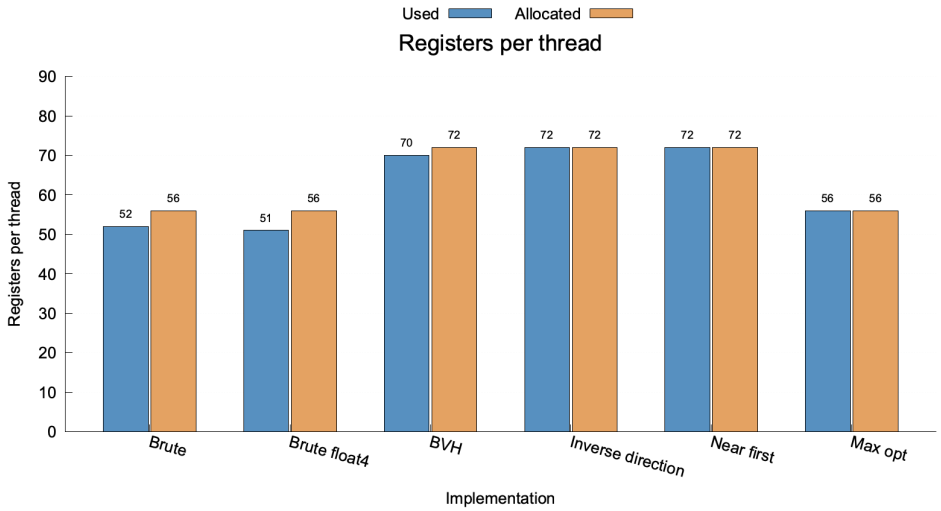


Figure 9: Register usage per kernel

5.7 Memory Behaviour

Memory performance depends on data layout, access locality, cache reuse, instruction width, and traversal dependencies. Optimized BVH reduced global-load requests from 45.79 million to 30.83 million (32.7%) and L2-to-L1 read traffic from 251.20 MB to 206.16 MB (17.9%).

The brute-force `float4` variant reduced global-load requests by 78.8% and runtime by 29.5%, demonstrating a clear layout benefit. Its L1 hit rate decreased from 94.68% to 92.69%; reducing accesses was more important than improving the hit percentage.

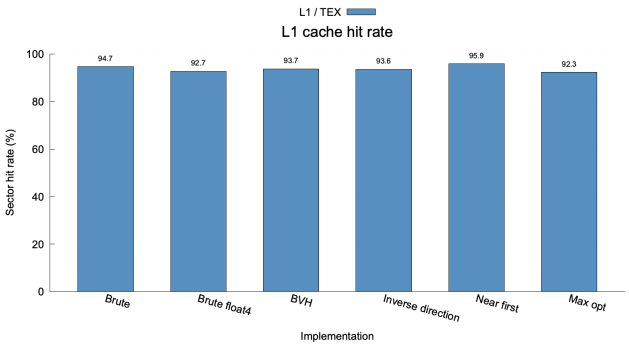


Figure 10: L1 Cache hits

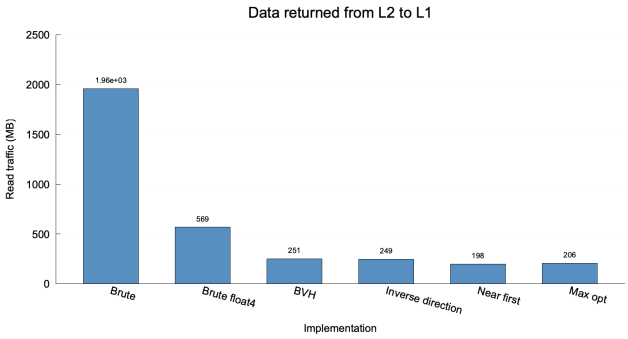


Figure 11: Traffic from L2 to L1

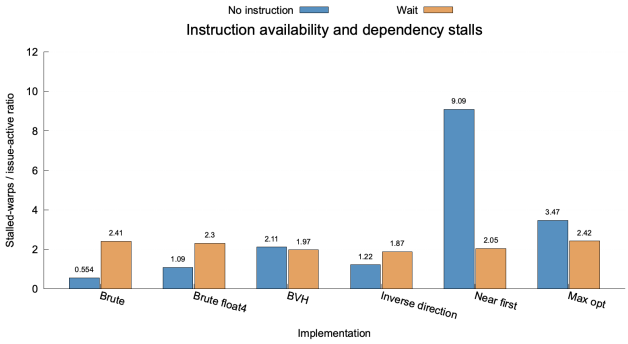


Figure 12: Instruction Dependencies

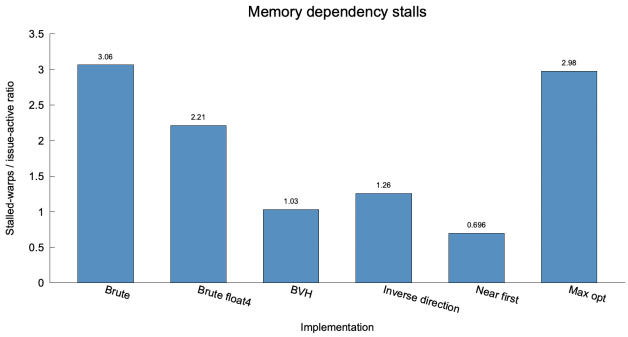


Figure 13: Memory Dependencies

5.8 Identified Bottlenecks

The strongest improvement came from reducing work: basic BVH reduced warp instructions by 89.0% relative to brute force and improved runtime from 82.430 ms to 8.248 ms in the earlier profiling session. The final benchmark session reported 83.463 ms and 8.315 ms for these same two implementations. Reusing inverse ray directions then reduced BVH instructions by 11.3% and runtime by 16.5% in that earlier session, with almost unchanged global-load request counts, supporting the arithmetic optimization. The later benchmark session measured an 18.1% reduction.

Standalone near-first traversal performed poorly: it executed 14.0% more instructions and took 14.487 ms, 75.6% longer than basic BVH. Its no-instruction stall ratio increased from 2.114 to 9.093, indicating instruction-availability problems that warrant investigation, rather than proving an instruction-cache bottleneck.

Optimized BVH achieved 3.735 ms, a 2.21× speedup over basic BVH. Its long-scoreboard stall ratio increased from 1.032 to 2.977, while lane participation remained low. These observations motivate examining memory dependencies and traversal coherence.

6 Optimizations

The experiments investigated three main opportunities: reducing the number of intersection tests through BVH traversal, reducing arithmetic and memory-access costs, and improving traversal decisions. Each standalone variant was compared with its corresponding baseline. The most optimized BVH combined several changes, so its performance cannot be attributed to one optimization alone.

Unless stated otherwise, results below use normal compilation without fast math. Kernel times are averages of three process runs, each containing five warm-up launches followed by twenty timed launches. Timing error bars represent the standard deviation of the three process averages.

6.0.1 Performance Comparison

The main observations were:

- BVH provided the largest improvement by avoiding exhaustive sphere testing.
- Brute-force `float4` storage and BVH inverse-direction reuse improved performance on both large scenes.
- The combined optimized BVH was the fastest implementation.
- Similar sphere counts did not imply similar BVH execution times: scene arrangement substantially affected performance.

For scene 5000, baseline brute force required 83.463 ms, while baseline BVH required 8.315 ms, giving a 10.04× speedup. Optimized BVH reduced this further to 3.748 ms, achieving a 22.27× speedup over brute force.

For scene 5001, baseline brute force required 77.677 ms, baseline BVH 5.513 ms, and optimized BVH 1.887 ms. The corresponding speedups over brute force were 14.09× and 41.16×. Although the sphere counts were almost identical, the different scene changed traversal work, ray coherence, and memory locality.

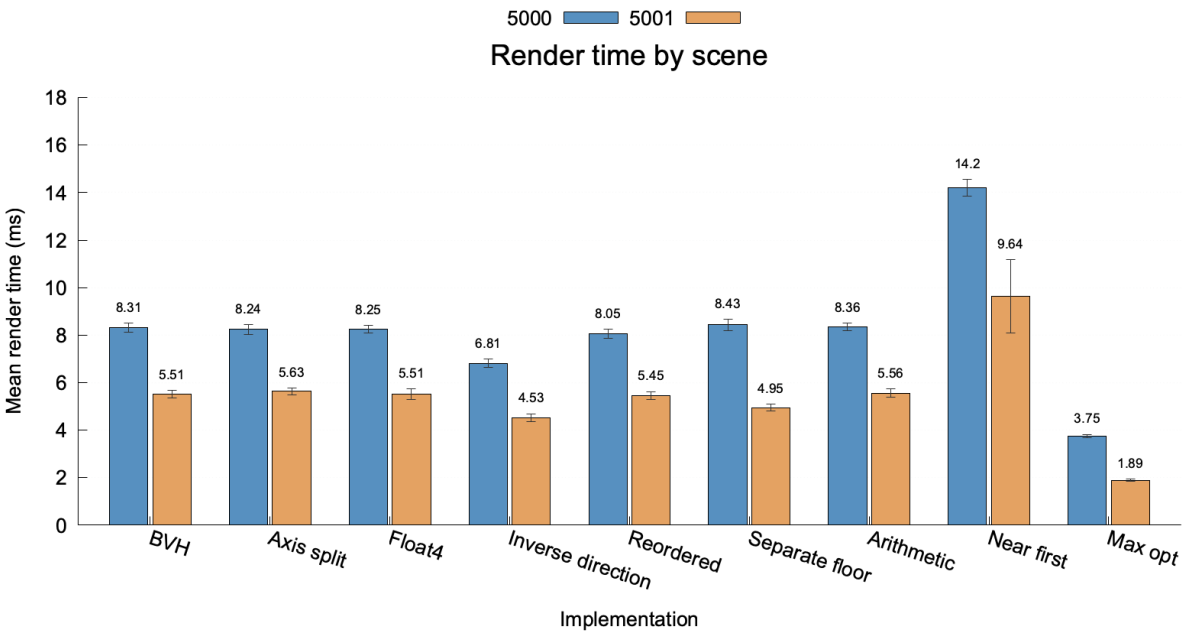


Figure 14: BVH implementation timings for scenes 5000 and 5001. Lower execution time is better.

6.0.2 BVH Optimizations

The BVH experiments examined:

- Leaf capacity and the choice of splitting axis.
- Reusing inverse ray directions during AABB intersection.
- Near-first traversal and avoiding redundant intersection tests.
- Scene reordering, compact sphere attributes, and separate handling of the floor.

An initial experiment tested the effect of leaf capacity on kernel latency. A maximum leaf capacity of one produced approximately 18% lower execution time than a capacity of sixteen for 5000 spheres. Smaller leaves reduce the number of sphere tests after reaching a leaf, but increase tree size and traversal work. The result therefore reflects a trade-off rather than a universal preference for the smallest possible leaves.

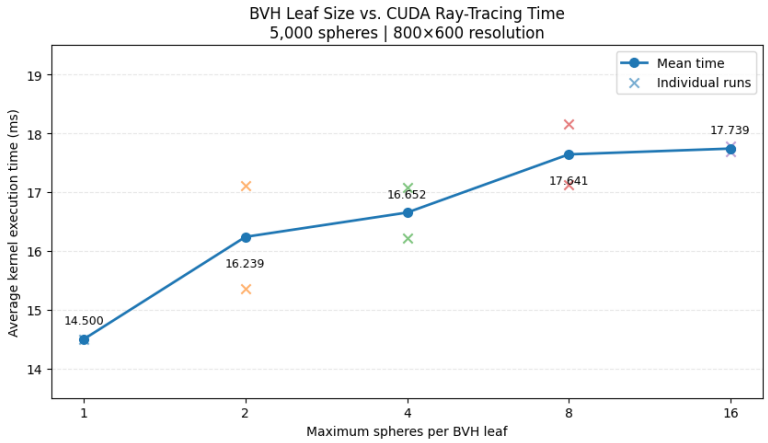


Figure 15: Effect of maximum BVH leaf capacity on kernel execution time for 5000 spheres at 800×600 resolution. Repeated measurements are averaged. This experiment used the earlier, largely unoptimized BVH.

Inverse-direction reuse was a consistent isolated improvement. Instead of repeatedly dividing by ray-direction components during AABB tests, the implementation computed their reciprocals once per ray and reused them through multiplication. Runtime decreased from 8.315 ms to 6.813 ms on scene 5000 and from 5.513 ms to 4.532 ms on scene 5001, reductions of 18.1% and 17.8%.

The optimized traversal visited the nearer child directly and stored only the deferred child. It reused child AABB intersection results, avoided redundant leaf AABB tests, and stored entry distances to discard deferred subtrees that could no longer contain a closer hit. These changes targeted unnecessary traversal work and stack operations.

Additional experiments changed the splitting axis from the longest centroid extent to the longest enclosing AABB extent, reordered sphere records into leaf order, and stored intersection attributes separately using `float4`. Their standalone timing improvements were inconsistent. Separating the large floor sphere from the BVH was more clearly scene-dependent: it provided no demonstrated benefit on scene 5000, but reduced runtime by 10.2% on scene 5001.

6.1 Other Optimization Attempts

Other experiments focused on:

- Compact sphere-attribute storage for brute force.
- Simplified sphere-intersection arithmetic and reciprocal square roots.
- Constant-memory storage of sphere attributes.
- Compilation with `--use_fast_math`.

The brute-force `float4` variant reduced runtime from 83.463 ms to 57.893 ms on scene 5000 and from 77.677 ms to 58.944 ms on scene 5001. These represent reductions of 30.6% and 24.1%. The scene 5000 profile also showed substantially fewer global-load requests and executed instructions, supporting compact attribute storage as a useful change for brute force.

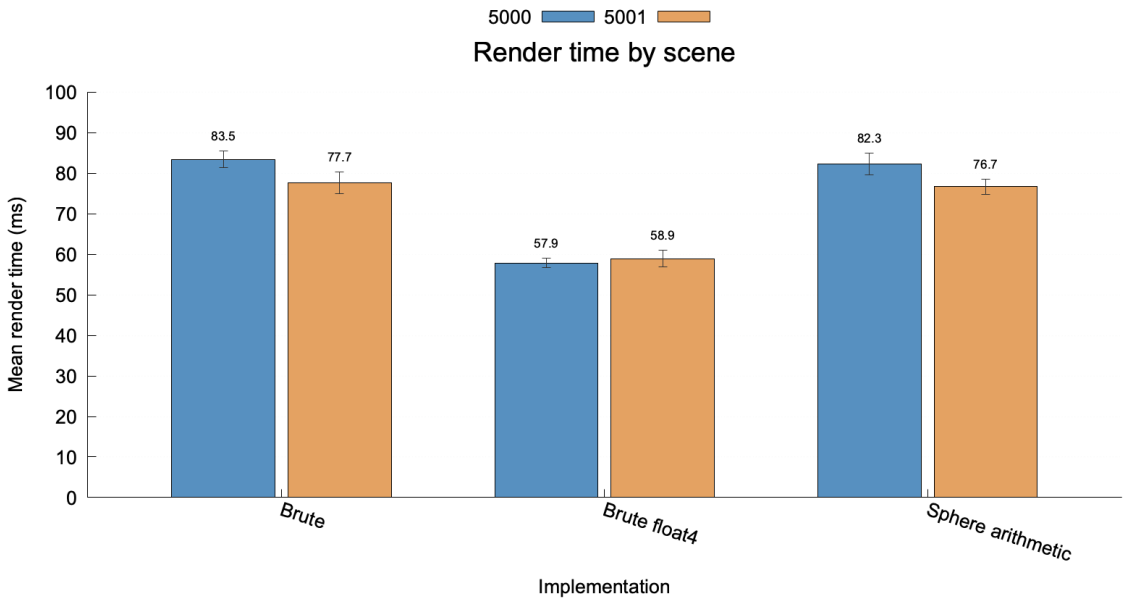


Figure 16: Brute-force implementation timings for scenes 5000 and 5001. Compact `float4` storage provides a larger benefit than the standalone arithmetic changes.

Arithmetic changes included the half-b form of the sphere intersection quadratic and `rsqrtf` for vector normalization. The optimized BVH also computed the quadratic coefficient $a = \mathbf{d} \cdot \mathbf{d}$ once per ray and passed it to intersection tests. This preserved support for directions whose length was not exactly one. The brute-force arithmetic variant improved mean runtime by only 1.4% on scene 5000 and 1.2% on scene 5001, small relative to the observed variability.

Constant-memory experiments stored sphere intersection attributes in a 4000-entry `float4` array occupying 64,000 bytes. Materials and BVH nodes remained in global memory; optimized BVH also retained its separately tested floor in global memory. The experiments used prefixes of scene 5000 containing 100, 1000, 2000, 3000, and 4000 spheres.

Constant storage reduced brute-force runtime at 100 spheres from 0.510 ms to 0.447 ms, a 12.4% improvement. However, the benefit did not persist for larger prefixes. All fifteen constant-memory renders were byte-identical to their corresponding parent renders.

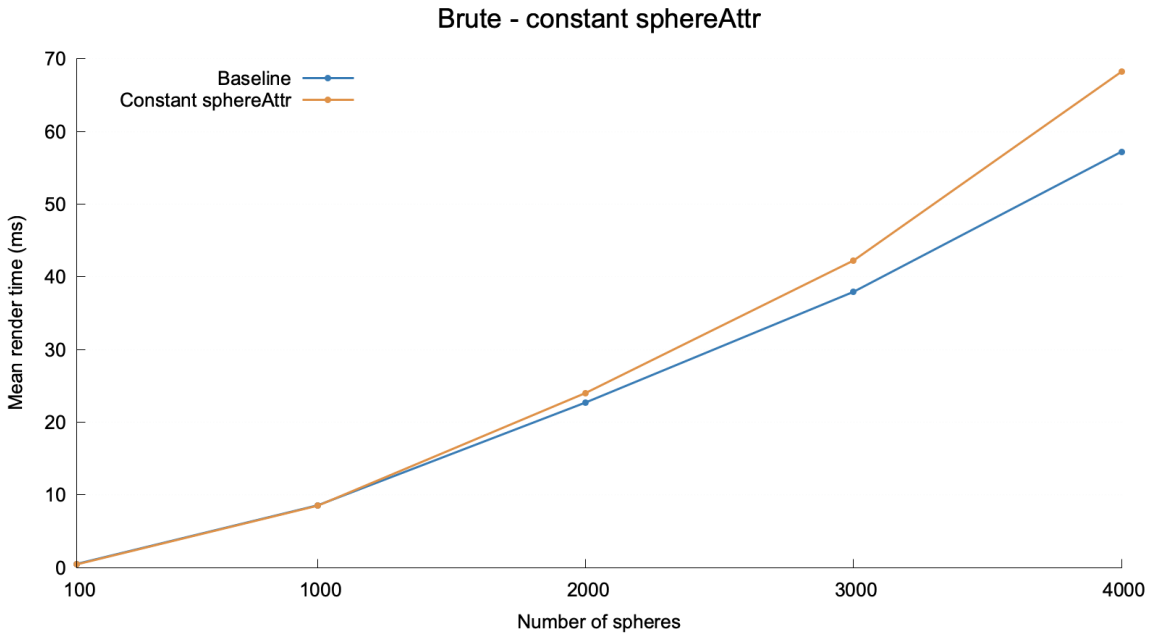


Figure 17: Brute-force runtime with baseline and constant `sphereAttr` storage. Constant memory did not provide a consistent benefit as the scene grew.

In the earlier fast-math experiment, optimized BVH improved from 3.735 ms to 3.703 ms on scene 5000, only approximately 0.9%, and from 1.898 ms to 1.691 ms on scene 5001, approximately 10.9%. These results suggest a scene-dependent benefit. Fast math also changed rendered pixels, so its speedup must be considered together with numerical accuracy. These measurements belong to the earlier experiment and are separate from the latest normal-compilation comparisons.

6.2 Unsuccessful Optimizations and Analysis

The unsuccessful or inconclusive attempts were:

- Standalone near-first traversal, which added more work than it eliminated.
- Constant-memory attributes for larger scenes.
- BVH attribute packing, and arithmetic changes, which did not show consistent gains.
- Scene reordering, whose small timing changes were not clearly separated from run-to-run variability.

Standalone near-first traversal increased execution time from 8.315 ms to 14.211 ms on scene 5000 and from 5.513 ms to 9.638 ms on scene 5001. It was therefore approximately 70.9% and 74.8% slower than basic BVH. On scene 5000, its executed warp-instruction count increased from 969 million to 1104 million. Additional child AABB tests and repeated tests introduced overhead. This was a useful failed experiment: just visiting the nearer child first did not help when the code still paid for additional and repeated box tests. The final optimized BVH kept near-first ordering, but also reused the child intersection results, avoided redundant leaf tests, and stored only the deferred child on the stack. The lesson is that traversal order by itself is not enough; the extra work needed to implement that order also matters.

Constant memory also regressed at larger sizes. At 4000 spheres, brute-force runtime increased by 19.2% and basic BVH runtime by 26.4%. Optimized BVH's mean increased by 42.4%, although one

unusually slow repeat inflated this result. Its median remained approximately 9.1% above the baseline median.

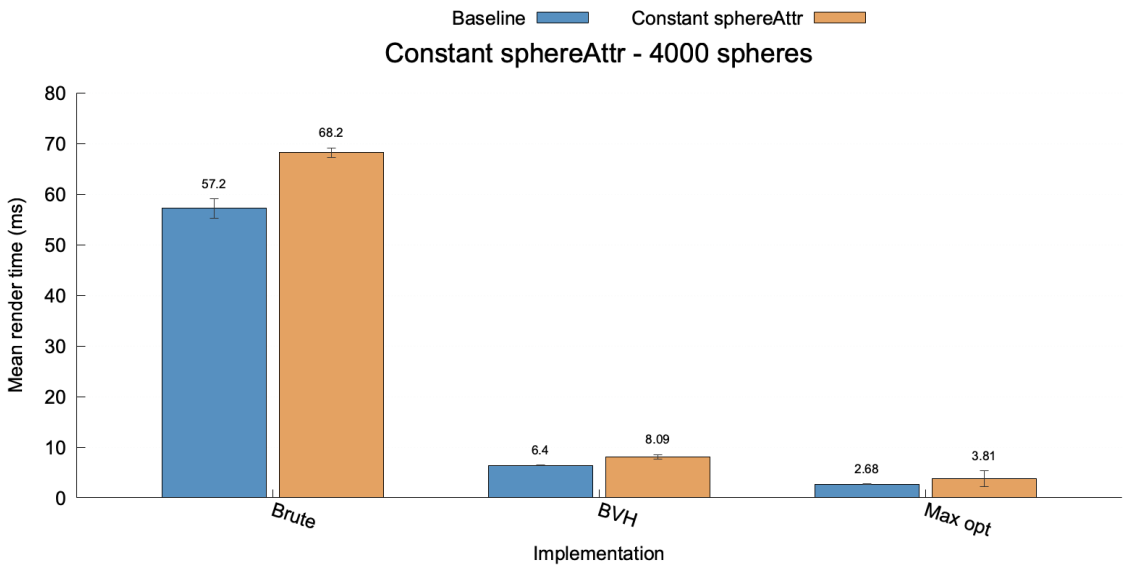


Figure 18: Constant `sphereAttr` storage at 4000 spheres. Error bars retain the variability, including the unusually slow optimized-BVH repeat.

Finally, standalone improvements cannot simply be added to predict the combined result. They are measured against the same baseline, their savings may overlap, and combining them can change register allocation, instruction scheduling, and memory behaviour.

7 Rendered Outputs

7.1 Baseline CUDA Rendering

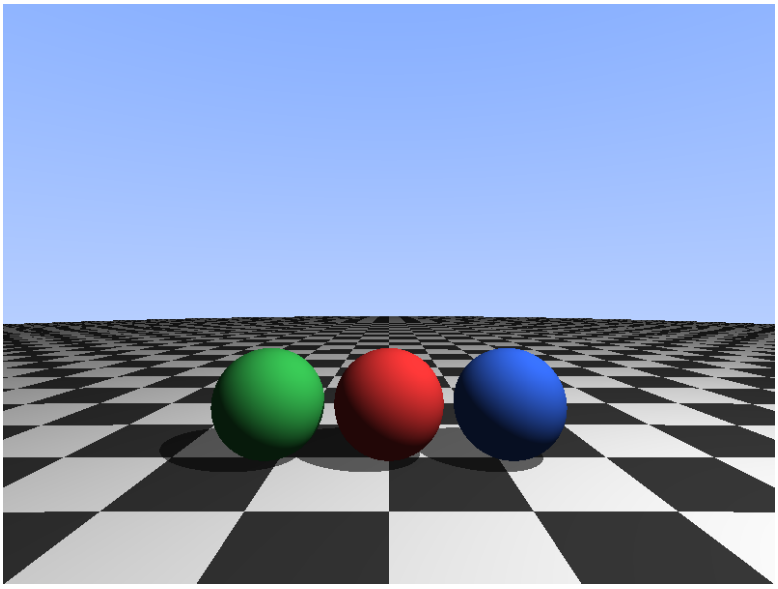


Figure 19: Baseline rendering of three coloured spheres above a checkerboard floor, showing surface shading and cast shadows.

7.2 Multi-Bounce Reflections

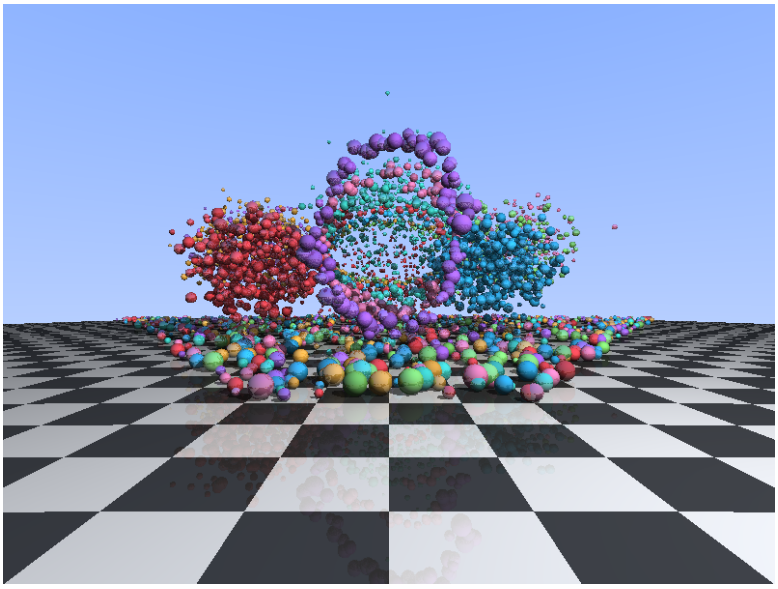


Figure 20: Rendering of the 5000-sphere scene. Identical for brute force and BVH

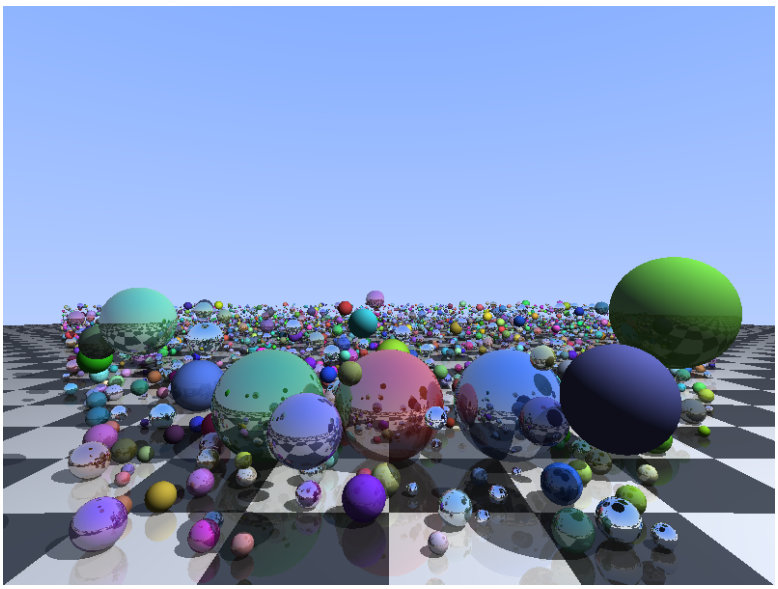


Figure 21: Rendering of the 5001 sphere scene. Identical for brute force and BVH

8 Conclusion

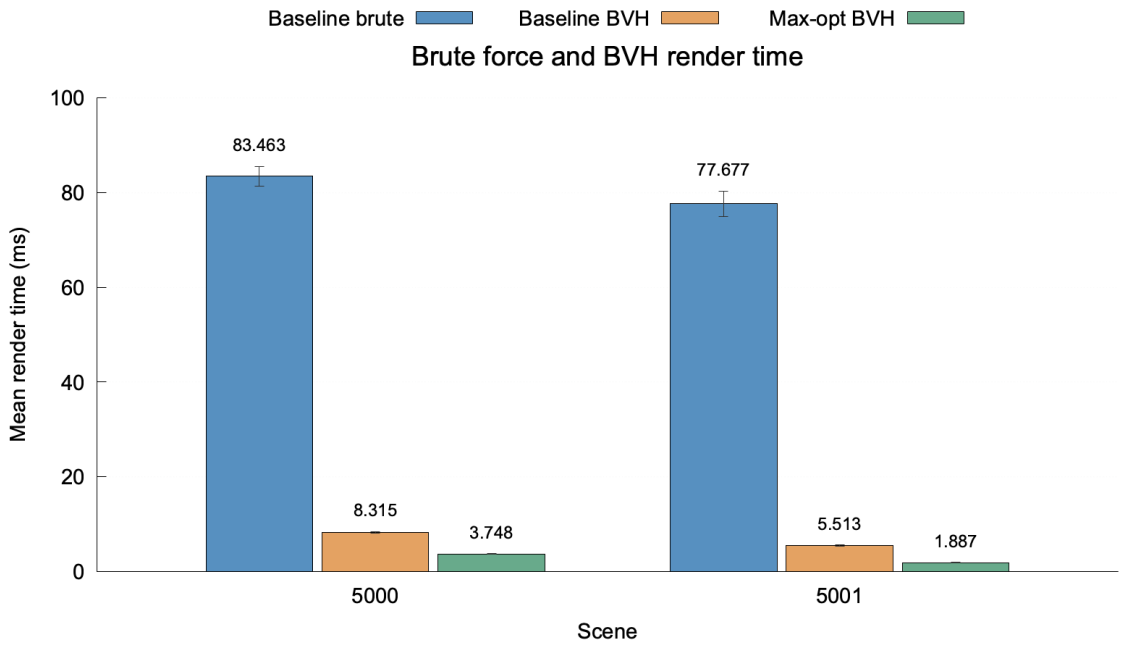


Figure 22: Baseline versus optimized BVH on the two large scenes.

Baseline BVH achieved 10.04× and 14.09× speedups over baseline brute force on scenes 5000 and 5001, respectively; max-opt BVH achieved 22.27× and 41.16× speedups on those same scenes.

Overall, optimized BVH reduced runtime from 8.315 ms to 3.748 ms on scene 5000, a 54.9% reduction and a 2.22× speedup over baseline BVH. On scene 5001, runtime decreased from 5.513 ms to 1.887 ms, a 65.8% reduction and a 2.92× speedup. The larger improvement on scene 5001 demonstrates that the effectiveness of the combined optimizations depends on the scene, even when sphere counts are almost identical. The profiles support this distinction: optimized BVH executes approximately 372 million warp instructions on scene 5000 but only 219 million on scene 5001, while average active threads per warp instruction increase from 9.22 to 12.85.

The main bottleneck was the amount of intersection and traversal work, not simply low occupancy or a lower cache hit percentage. BVH gave the largest improvement because it avoided testing every sphere for every ray. Even when BVH had fewer active threads per warp instruction, its much lower instruction count more than made up for that loss. Higher occupancy was useful, but did not by itself explain the runtime gains.

The failed attempts were informative too. Standalone near-first traversal made the kernel slower, while near-first traversal combined with avoiding redundant AABB tests and better stack handling contributed to the final optimized version. Constant-memory storage also did not give a consistent improvement as scene size increased. These experiments showed that an optimization only helps if the work it saves outweighs the extra work it introduces.

All speedups reported here are based on CUDA-event kernel timings. They do not measure total end-to-end rendering time, which would also include scene and BVH construction, memory allocation and transfers, and writing the output image.